

Adv C Module

1. MCQ

1. What is the output of the following code?

```
int a = 5;  
printf("%d", a++ + ++a);
```

 - a. 10
 - b. 11
 - c. 12
 - d. Undefined behavior
2. Which keyword avoids compiler optimization?
 - a. Static
 - b. Extern
 - c. Volatile
 - d. register
3. Which operator has highest precedence?
 - a. +
 - b. *
 - c. =
 - d. ()
4. What is the output of the following code?

```
printf("%d", sizeof("HELLO"));
```

 - a. 4
 - b. 5
 - c. 6
 - d. Compiler-dependent
5. What is the purpose of static keyword inside a function?
 - a. Faster execution
 - b. Retains value between calls
 - c. Allocates heap memory
 - d. Makes variable global
6. What is the output of the following code?

```
int x = 0;  
if(x = 1)  
    printf("YES");  
else  
    printf("NO");
```

- a. YES
 - b. NO
 - c. Error
 - d. Undefined
7. Which function allocates memory dynamically in C programming?
- a. `alloc()`
 - b. `malloc()`
 - c. `memalloc()`
 - d. `new()`
8. Which loop executes at least once?
- a. `for`
 - b. `while`
 - c. `do-while`
 - d. infinite
9. What is the output of the following code?
- ```
int i;
for(i=0; i<3; i++);
 printf("%d", i);
```
- a. 2
  - b. 3
  - c. 0
  - d. Error
10. What is a dangling pointer?
- a. Uninitialized pointer
  - b. Pointer to freed memory
  - c. Pointer to stack
  - d. NULL pointer
11. Which header file contains `malloc()`?
- a. `stdio.h`
  - b. `stdlib.h`
  - c. `string.h`
  - d. `Memory.h`

12. What is the output of the following code?
- ```
static int i;  
printf("%d", i);
```
- a. Garbage
 - b. Error
 - c. 0
 - d. Undefined
13. What is undefined behavior?
- a. Compile error
 - b. Predictable output
 - c. Compiler-dependent unpredictable behavior
 - d. Runtime crash always
14. What is the default return type of main() function?
- a. void
 - b. int
 - c. char
 - d. long
15. What is the output of the following code?
- ```
int a = 10;
int *p = &a;
*p = 20;
printf("%d", a);
```
- a. 10
  - b. 20
  - c. Garbage
  - d. Error
16. What does extern do?
- a. Allocates memory
  - b. Declares variable defined elsewhere
  - c. Makes variable constant
  - d. Restricts scope
17. Best data type for bit manipulation:
- a. float
  - b. double
  - c. int
  - d. Char

18. Stack overflow occurs due to:
- a. Heap memory
  - b. Infinite recursion
  - c. Pointer arithmetic
  - d. Static allocation
19. What is the output of the following code?
- ```
int a = 3, b = 4;  
printf("%d", a & b);
```
- a. 0
 - b. 1
 - c. 4
 - d. 7
20. Purpose of header guards:
- a. Faster code
 - b. Prevent multiple inclusion
 - c. Reduce memory
 - d. Enable linking
21. NULL represents:
- a. 0
 - b. Empty string
 - c. Invalid pointer
 - d. Garbage
22. Segmentation fault occurs due to:
- a. Syntax error
 - b. Invalid memory access
 - c. Logic error
 - d. Infinite loop
23. What is the output of the following code?
- ```
int x = 10;
printf("%d", x / 3);
```
- a. 3
  - b. 3.33
  - c. 4
  - d. Error

24. Which is NOT a storage class in C?
- a. auto
  - b. register
  - c. static
  - d. Globa
25. What is the output of the following code?
- ```
printf("%c", 65);
```
- a. 65
 - b. A
 - c. a
 - d. error
26. In C, array index starts from:
- a. 0
 - b. 1
 - c. -1
 - d. Compiler-dependent
27. Which of the following is non-volatile memory?
- a. RAM
 - b. Cache
 - c. Flash
 - d. Register
28. Predict the output of the following code:
- ```
#include <stdio.h>
int main(void)
{
 int x = 5;
 printf("%d %d %d", x++, x++, ++x);
 return 0;
}
```
29. Predict the output of the following code:
- ```
#include <stdio.h>
void func(int x)
{
    static int y = 0;
    y += x;
    printf("%d ", y);
}
```

```

int main(void)
{
    func(5);
    func(5);
    func(5);
    return 0;
}

```

30. Predict the output of the following code:

```

#include <stdio.h>
int main(void)
{
    char arr[] = "embedded";
    char *p = arr;
    printf("%c %c %c", *p, *(p+3), *(arr+5));
}

```

31. Predict the output of the following code:

```

#include <stdio.h>
int main(void)
{
    printf("%lu %lu", sizeof(int*), sizeof(char*));
    return 0;
}

```

32. Predict the output of the following code:

```

#include <stdio.h>
void swap(int *a, int *b)
{
    int *temp = a;
    a = b;
    b = temp;
}
int main(void)
{
    int x = 10, y = 20;
    swap(&x, &y);
    printf("%d %d", x, y);
    return 0;
}

```

33. Predict the output of the following code:

```
#include <stdio.h>
int main(void)
{
    int a = 5;
    int * const p = &a;
    *p = 10;
    printf("%d", a);
    return 0;
}
```

34. Predict the output of the following code:

```
#include <stdio.h>
int main(void)
{
    int a = 0;
    if(a = a + 1)
        printf("TRUE ");
    if(a == 2)
        printf("FALSE");
    return 0;
}
```

2. Subjective

2.1 Basic Refreshers

1. What is type conversion? Explain implicit and explicit conversion.
2. What causes a segmentation fault?
3. Explain bitwise operations with examples.
4. What is the range of an int data type?
5. What is the difference between pass by value and pass by reference.
6. What is the difference between pre-increment and post-increment?

2.2 1D Pointers and Functions

1. Explain pointers and different types of pointers.
2. Explain pointer arithmetic.
3. Is pointer arithmetic allowed on void *? Why?
4. What is a constant pointer?
5. What is a pointer to a constant?

2.3 Storage classes and memory segments

1. Explain the different memory segments.
2. What are storage classes? Explain with examples.
3. What is a static variable? Where is it used?
4. Explain global, extern, and register keywords with examples.
5. What is the difference between heap and stack.

2.4 2D Pointers and DMA

1. What is dynamic memory allocation? Explain malloc, calloc, realloc, and free.
2. What is a function pointer? What are its advantages?
3. What is the difference between array of pointers and pointer to an array.
4. When to use 2D arrays vs array of pointers.
5. What is a null pointer?
6. What is a void pointer?
7. What happens if you dereference a null pointer?
8. Explain constant pointer vs pointer to constant.
9. Explain dangling pointers and wild pointers.
10. Draw and explain memory layout.
11. Explain dynamic memory allocation using single and double pointers.
12.

```
int fun(int *p) {  
    p = malloc(sizeof(int));  
    *p = 100;  
    return *p;  
}
```

 What is the output of the above code.

2.5 Preprocessing

1. What are compilation stages in C?
2. Explain optimization done by the compiler/CPU.

2.6 UDT

1. Explain structure vs union.
2. What is the differentiate between macro and inline function.
3. What is typedef? Where is it used?
4. What is structure padding?
5. Explain bit-field structures.
6. Calculate the size of a given structure and explain why.
7. What is the difference between structure and class.
8. How can structure alignment be optimized to reduce memory usage?
9. struct payload {
 char a;
 int len;
 char payload[10];
}; What is the size of this structure.

2.7 Miscellaneous

1. What is the volatile keyword? Why is it important in embedded systems?

2.8 Project

1. Explain the AddressBook project.
2. What is the time complexity for searching a name in an address book?
3. How is data stored in the address book project?
4. Explain validations for search, add, and edit operations.
5. Write logic for searching a contact.
6. Explain the MP3 project.
7. Explain LSB steganography.
8. Why is LSB used instead of MSB?
9. Why is data not encoded in the image header?
10. Explain encoding and decoding logic with code.

3. Programming

1. WAP to check whether two numbers are equal using bitwise.
2. Write a program to convert a string to uppercase.
3. WAP to set, clear, and toggle a bit using bitwise operators.
4. Write a program to find duplicate elements in an array.
5. Write a program to copy one string to another.

6. Write a program to swap two numbers without using a third variable.
7. WAP to reverse the string.
8. Write a program to reverse an array.
9. Write a program to reverse a number.
10. WAP to find the second largest element in an array.
11. Write a program to find whether two strings are anagrams.
12. WAP to check the given number is palindrome or not.
13. WAP to implement the strcmp, strcat and strchr.
14. WAP to remove the numbers from a string.

MC Module

1. Basics

1. Find the odd one out:
 - a. UART
 - b. SPI
 - c. I2C
 - d. USB
2. Which keyword makes a variable thread-safe in embedded ISR?
 - a. static
 - b. extern
 - c. volatile
 - d. register
3. What is the purpose of a bootloader?
 - a. Debug code
 - b. Load application firmware
 - c. Power management
 - d. Task scheduling
4. An interrupt is:
 - a. Polling method
 - b. Event-driven mechanism
 - c. Delay function
 - d. Timer

5. Which communication protocol is synchronous?
 - a. UART
 - b. SPI
 - c. RS232
 - d. CAN
6. What is a watchdog timer used for?
 - a. Generate clock
 - b. Reset system on fault
 - c. Store data
 - d. Control GPIO
7. What is PWM used for?
 - a. Data transfer
 - b. Speed/brightness control
 - c. Memory access
 - d. Clock sync
8. Which is the fastest memory?
 - a. Flash
 - b. RAM
 - c. Cache
 - d. EEPROM
9. What is a pull-up resistor used for?
 - a. Increase current
 - b. Avoid floating input
 - c. Boost voltage
 - d. Reduce noise
10. What does a microcontroller integrate?
 - a. CPU only
 - b. CPU + peripherals
 - c. External RAM only
 - d. GPU
11. Debouncing is related to:
 - a. Timers
 - b. Switches
 - c. Memory
 - d. ADC

12. Clock frequency affects:
 - a. Power only
 - b. Speed only
 - c. Speed & power
 - d. Memory
13. Which is an input device?
 - a. LED
 - b. Motor
 - c. Switch
 - d. Relay
14. Which memory stores firmware?
 - a. RAM
 - b. Cache
 - c. Flash
 - d. Register
15. What is a microcontroller?
16. What is the differentiate between microcontroller and microprocessor.
17. What is the difference between 8-bit, 16-bit, and 32-bit controllers.
18. What is the difference between RISC and CISC architecture.
19. What happens when the reset button is pressed?
20. Steps to create a program for a microcontroller using MPLAB IDE.
21. How will you interface an LED with a microcontroller?
22. How will you use any new microcontroller board?
23. Explain bootloader.
24. Are you familiar with SoC (System on Chip)?

2. Interrupts

1. Explain timers in microcontrollers.
2. Explain interrupts.
3. What happens internally when an interrupt occurs?
4. Explain interrupt latency and interrupt response time.
5. What is ISR?
6. Explain saving and restoring context during interrupts.
7. Explain ALU and its working.
8. What is polling?

3. Projects

1. What is the speed range used in your CAN project?
2. What is the message frequency in your CAN project?
3. Explain the CAN project.
4. Explain the baud rate used and why.

4. ADC

1. Explain ADC in detail.
2. What is ADC resolution?
3. What is the difference between 10-bit and 12-bit ADC.
4. How do you check whether ADC is working or not?
5. Explain how sensor values are converted into digital data in a microcontroller.

5. Protocols

1. Explain UART communication.
2. Explain I2C protocol.
3. Explain SPI protocol.
4. Explain CAN protocol.
5. What is the difference between CAN and CAN protocol.
6. Why are multiple nodes used in CAN instead of a single node?
7. Advantages of CAN over other protocols.
8. How do you decide which communication protocol to use?
9. Why are differential cables used?
10. Why is delay used in CAN communication?
11. How will you design your own protocol to communicate between nodes?
12. Why is SPI faster than I2C?
13. Explain half-duplex and full-duplex communication.
14. Which protocols use which transmission mode?
15. How does data transfer happen in SPI?
16. Explain the role of MISO and MOSI.

6. PWM

1. What is PWM (Pulse Width Modulation)? How is it used to control speed or brightness?
2. What is duty cycle in PWM? How does changing the duty cycle affect the output?

CPP Module

1. What is the difference between C and C++?
2. Explain OOP concepts in C++ (encapsulation, inheritance, polymorphism, abstraction).
3. What is a class and an object?
4. What is a constructor? Explain its types.
5. What is a destructor? When is it called?
6. What is function overloading?
7. What is operator overloading? Give an example.
8. What is inheritance? Explain its types.
9. Explain vectors in C++.
10. How do you check memory allocation failure in C++?
11. What is polymorphism? Difference between compile-time and run-time polymorphism.
12. What are virtual functions and why are they needed?
13. What is the difference between class and structure.
14. What is the difference between private and protected.
15. What is the difference between OOPS and POP?
16. What is the difference between function overloading and function overriding?
17. Why is const needed in C/C++?
18. Explain stack memory vs heap memory.
19. What is the difference between static memory and dynamic memory?

Ds Module

1. Basics

1. Find increasing elements in an array in $O(n)$.
2. Find the next greater element in $O(n)$.
3. Odd one out:
 - a. Stack
 - b. Queue
 - c. Array
 - d. Loop
4. Which grows fastest?
 - a. $O(n)$
 - b. $O(n \log n)$
 - c. $O(n^2)$
 - d. $O(\log n)$

2. Linked lists

1. What is a linked list? How is it different from an array?
2. Explain the types of linked lists.
3. Write an algorithm to insert a node at the end of a singly linked list.
4. How do you delete a node from a linked list?
5. What are the advantages and disadvantages of linked lists?
6. What is the difference between DFS and BFS?
7. Write a program to sort a linked list.
8. Find two elements in an array whose sum equals a given key in $O(n)$.

3. Stack

1. What is a stack? Explain the LIFO principle.
2. What are the basic stack operations?
3. Implement a stack using an array.
4. Implement a stack using a linked list.
5. What are real-time applications of stacks?

4. Searching and Sorting Techniques

1. What is searching? Why is it required?
2. Explain linear search with time complexity.
3. Explain binary search with time complexity.
4. What is the prerequisite for binary search?
5. What is the difference between linear search and binary search.

5. Queue

1. What is a queue? Explain the FIFO principle.
2. What are the basic operations of a queue?
3. What is a circular queue? Why is it needed?
4. Implement a queue using an array.
5. What are the applications of queues?

6. Hashing

1. What is hashing?
2. What is a hash function?
3. What is collision in hashing? How can it be resolved?
4. Explain open addressing and chaining techniques.
5. What are the advantages of hashing?

7. Trees

1. What is a tree data structure?
2. What is a binary tree?
3. What is a binary search tree (BST)?
4. Explain tree traversal techniques.
5. What are the applications of trees?

8. Project

1. Explain the APC project.
2. Explain the inverted search project.

LI Module

1. Basics

1. What does the ps command show?
 - a. Files
 - b. Processes
 - c. Memory
 - d. Users
2. What does dmesg command show?
 - a. User logs
 - b. Kernel logs
 - c. App logs
 - d. Network logs
3. What does the top command show?
 - a. Disk usage
 - b. Memory map
 - c. Running processes
 - d. Kernel config
4. What is the SIGKILL signal number?
 - a. 2
 - b. 9
 - c. 15
 - d. 18
5. Why is Embedded Linux preferred?
 - a. Cost
 - b. Flexibility
 - c. GUI
 - d. Size only
6. How does a hard link differ from a soft link?
 - a. Inode
 - b. Name
 - c. Permission
 - d. Size

7. What does ls -l command show?
 - a. Hidden files
 - b. Permissions
 - c. File size only
 - d. Processes
8. What does cross-compilation mean?
 - a. Compile twice
 - b. Compile on target
 - c. Compile for different architecture
 - d. Compile kernel
9. What is a shell?
 - a. Kernel
 - b. Hardware
 - c. Command interface
 - d. Driver
10. What is init/systemd responsible for?
 - a. Boot process
 - b. File creation
 - c. User login
 - d. Networking
11. What is the difference between static linking and dynamic linking.
12. Explain physical memory and virtual memory.

2. System call

1. What is a system call?
2. Why are system calls required in an operating system?
3. Explain the types of system calls with examples.
4. What is the difference between a system call and a library function?
5. Explain the booting sequence in detail.
6. What is GRUB?

3. Networking

1. What is computer networking?
2. Explain the OSI model layers.
3. What is the difference between TCP and UDP.
4. What is an IP address? Explain IPv4 and IPv6.
5. What is a port number? Why is it required?

4. Process

1. What is a zombie process?
2. How do you create and manage processes?
3. What is fork()?
4. What is a process crash?
5. What is a daemon process?

5. IPC

1. What is IPC? Why is it needed?
2. Explain shared memory IPC mechanism.
3. Explain message queues.

6. Signal

1. What is a signal in Linux?
2. Explain SIGKILL and SIGTERM.
3. How do you handle signals in C?
4. What is signal masking?
5. What happens when a signal is not handled?
6. Explain SIGSEGV and SIGINT signals.

7. Socket

1. What is a socket?
2. Explain the socket programming model.
3. What is the difference between TCP sockets and UDP sockets.
4. Explain bind(), listen(), and accept() functions.
5. What is a client-server model?

8. Threads

1. What is the difference between a process and a thread.
2. Explain mutex and semaphore.
3. What is the difference between mutex and semaphore.
4. Explain thread creation and mutex usage.
5. What is a thread crash?
6. What is context switching?
7. What is a deadlock?
8. What are the necessary conditions for a deadlock?

9. Project

1. Explain your Mini Shell project.
2. Explain the TFTP project.

10. Process and Memory Management

1. What is memory management in an operating system?
2. What is the difference between physical memory and virtual memory.
3. What is paging?
4. What is segmentation?
5. What is a page fault?

General Questions Aptitude

1. Explain shell scripting.
2. How does a script know whether it is a shell or bash script?
3. What is a shebang (!) statement?
4. What happens if bash is replaced with python in the shebang line?
5. How do you debug Linux errors when a script produces no output?
6. Explain the use of: cp, mv, mkdir, chmod, chmod +x, chown, ps, ip, version, scp, sed.
7. Write a command to copy contents from one file to another.
8. Explain your experience in your previous company.
9. What roles and responsibilities did you handle?
10. Why did you choose embedded systems?
11. Explain the challenges you faced in your projects and how you overcame them.
12. What do you know about Zentree?
13. Do you have any questions for us?
14. What role are you expecting?
15. Why did you apply for this position?
16. What are your thoughts on embedded systems?

17. What do you know about our company (LG) and the domains we work in?
18. What challenges did you face in your projects, and how did you overcome them?
19. Explain the undo–redo system implementation.
20. How do you find a specific string in a file in Linux?
21. How do you find a running process and kill it in Linux?
22. What is the difference between nice and renice?
23. What is partitioning in Linux?
24. If the disk is full, how will you analyze the issue?
25. If the system is slow, how will you troubleshoot it?
26. What is the difference between BIOS and firmware?
27. Why is Windows more popular for desktop users?
28. What is the difference between Windows and Linux?
29. What is the difference between a list and a tuple?
30. What is a decorator in Python?
31. Explain break, continue, and pass.
32. What is the difference between is and ==?
33. What loops are available in Python?
34. What is the difference between mutable and immutable objects?